

CRM Sync — Clean Room Utility & Security Rules

The **Cloudflare-native clean room**: how CRM Sync joins and activates customer data under GDPR/CCPA/CPRA rules on D1, R2, and Durable Objects — privacy-safe data collaboration on the network the worker already runs on.

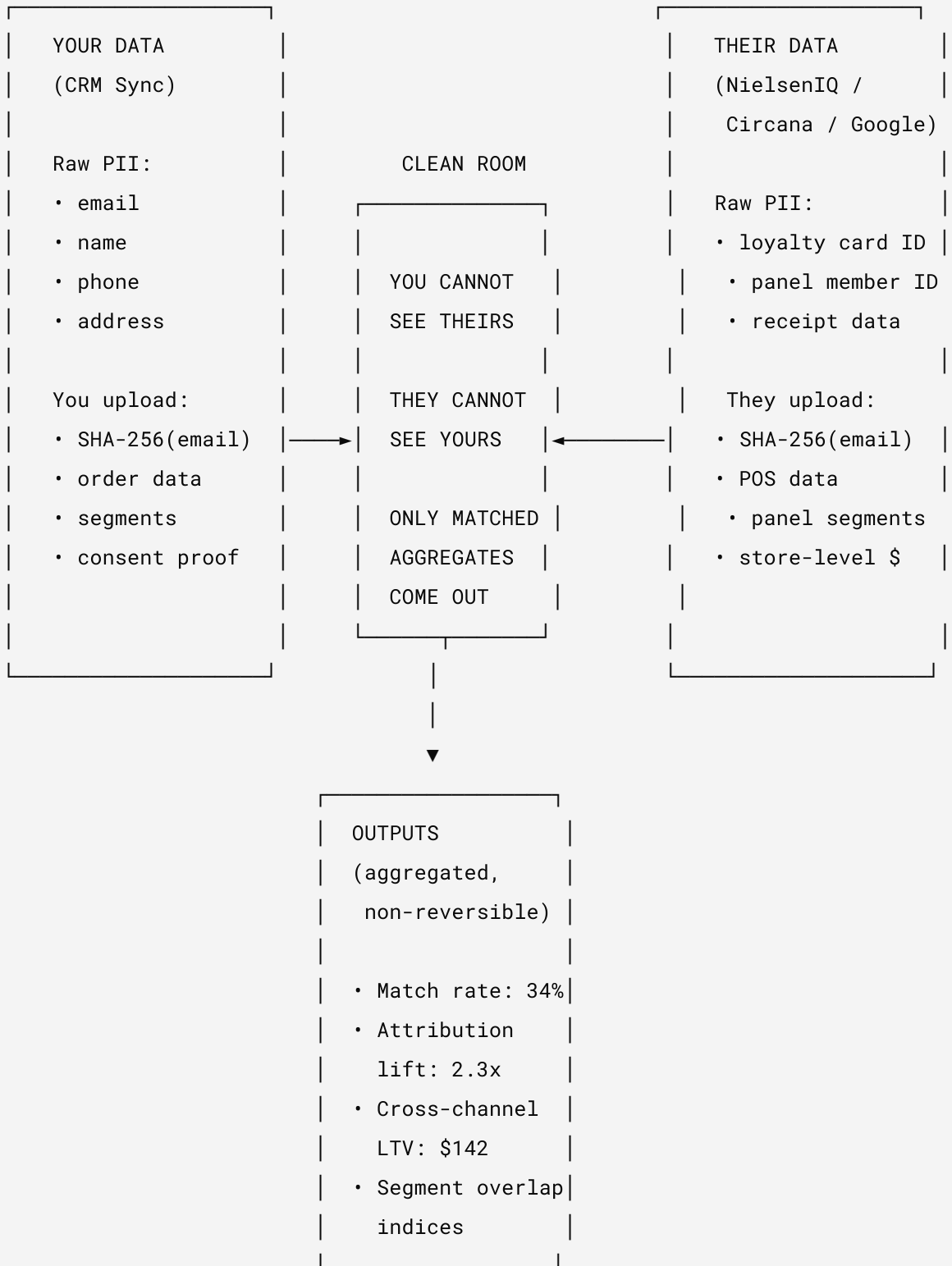
Version: 1.1 – Cloudflare-Native Architecture Date: 2026-05-27 Classification: Public · Spec

Compliance: GDPR Art. 6/9, CCPA §1798.140, CPRA, UK DPA 2018

Infrastructure: Cloudflare D1 + R2 + Durable Objects (same network as Hydrogen/Worker)

1. WHAT IS A CLEAN ROOM

A data clean room is a **controlled environment where two or more parties match their datasets without either party seeing the other's raw data.**



Key property: Neither party can reverse-engineer the other's raw data. Only pre-agreed aggregate statistics come out.

2. WHY YOU NEED ONE

Without a clean room, connecting D2C data to retail measurement requires sharing raw PII — which violates:

REGULATION	VIOLATION	PENALTY
GDPR Art. 6	No lawful basis for sharing raw email with third party for profiling	Up to €20M or 4% global revenue
CCPA §1798.140	"Sale" of personal information without opt-out mechanism	\$7,500 per intentional violation
CPRA	Sharing for cross-context behavioral advertising without consent	\$7,500 per record

A clean room makes the same matching possible **without sharing raw PII**.

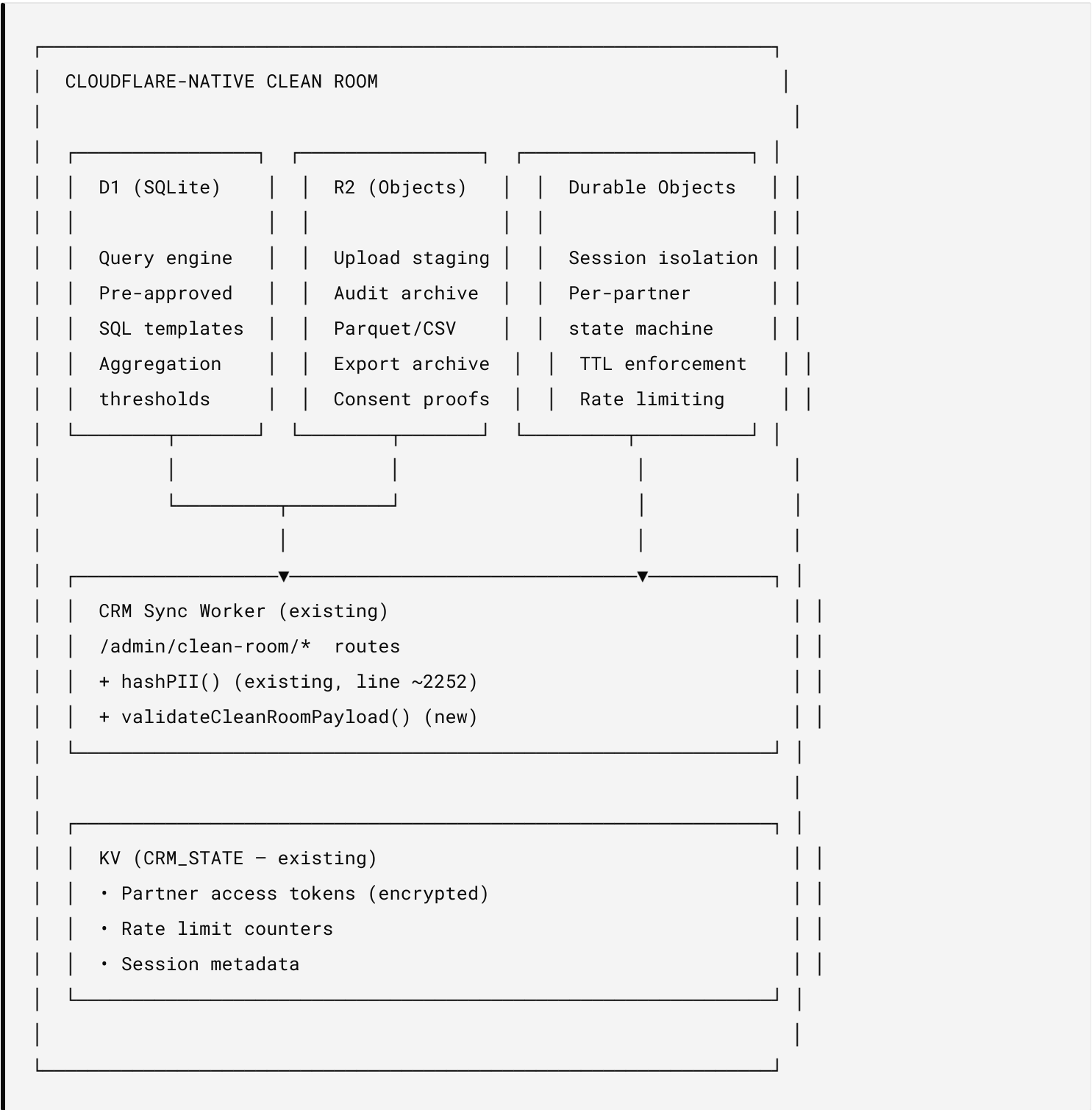
3. ARCHITECTURE: CLOUDFLARE-NATIVE CLEAN ROOM

CRM Sync implements a **self-hosted clean room** on Cloudflare's edge infrastructure — the same network that hosts the Shopify Hydrogen storefront (Oxygen Workers) and the CRM Sync worker. No AWS, Snowflake, or third-party clean room service required.

3.1 Why Cloudflare-Native

FACTOR	EXTERNAL CLEAN ROOM (AWS/SNOWFLAKE)	CLOUDFLARE-NATIVE
Data residency	Data leaves your infrastructure	Data stays on your Cloudflare account
Cost	AWS compute + storage per query	D1: free tier 5M reads/day; R2: free egress
Network	Cross-cloud hops (CF → AWS → CF)	Same edge network as Hydrogen + Worker
Latency	Partner-dependent setup time	Instant — same-stack queries
Control	Governed by provider's policies	You own the governance rules entirely
Multi-tenant	Separate accounts per tenant	Same D1 database, tenant-scoped queries

3.2 Component Mapping



3.3 wrangler.toml Bindings

```
# D1 - Clean room query engine (SQLite at the edge)
[[d1_databases]]
binding = "CLEAN_ROOM_DB"
database_name = "crm-clean-room"
database_id = "..." # Set after `wrangler d1 create crm-clean-room`

# R2 - Upload staging + audit archive (already used for analytics exports)
[[r2_buckets]]
binding = "ANALYTICS_EXPORTS"
bucket_name = "crm-analytics-exports"

# Durable Objects - Per-partner session isolation
[durable_objects]
bindings = [
  { name = "CLEAN_ROOM_SESSION", class_name = "CleanRoomSession" }
]

[[migrations]]
tag = "v1"
new_classes = ["CleanRoomSession"]
```

3.4 D1 Schema

```
-- Partner uploads (their hashed data loaded into D1 for matching)
CREATE TABLE IF NOT EXISTS partner_uploads (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  session_id TEXT NOT NULL,
  partner TEXT NOT NULL,          -- "nielseniq", "circana", "google_adh"
  hashed_email TEXT NOT NULL,
  attributes TEXT,                -- JSON: POS data, panel segments, etc.
  uploaded_at TEXT DEFAULT (datetime('now')),
  expires_at TEXT NOT NULL,       -- Auto-delete after 90 days
  UNIQUE(session_id, partner, hashed_email)
);

-- CRM uploads (our hashed data, loaded per-session)
CREATE TABLE IF NOT EXISTS crm_uploads (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  session_id TEXT NOT NULL,
  hashed_email TEXT NOT NULL,
  segments TEXT,                  -- JSON array
  order_data TEXT,                -- JSON: order_count_30d, order_total_30d, etc.
  channel_source TEXT,
  has_mandate INTEGER DEFAULT 0,
  consent_scope TEXT NOT NULL,
  consent_granted_at TEXT NOT NULL,
  uploaded_at TEXT DEFAULT (datetime('now')),
  UNIQUE(session_id, hashed_email)
);

-- Query results (aggregated outputs only)
CREATE TABLE IF NOT EXISTS query_results (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  session_id TEXT NOT NULL,
  query_template TEXT NOT NULL,   -- "attribution_lift", "segment_overlap", etc.
  result TEXT NOT NULL,           -- JSON: aggregated, non-PII output
  computed_at TEXT DEFAULT (datetime('now')),
  match_rate REAL,
  records_matched INTEGER,
  aggregation_threshold_met INTEGER DEFAULT 1
);
```

```
-- Indexes for performance
CREATE INDEX IF NOT EXISTS idx_partner_session ON partner_uploads(session_id);
CREATE INDEX IF NOT EXISTS idx_partner_email ON partner_uploads(hashed_email);
CREATE INDEX IF NOT EXISTS idx_crm_session ON crm_uploads(session_id);
CREATE INDEX IF NOT EXISTS idx_crm_email ON crm_uploads(hashed_email);
CREATE INDEX IF NOT EXISTS idx_partner_expires ON partner_uploads(expires_at);
```


3.5 Durable Object: CleanRoomSession

```
// Each clean room analysis runs inside an isolated Durable Object
// Partner = NielsenIQ, Circana, or Google ADH
// Session = one analysis window (e.g., "attribution_lift for May 2026")
```

```
export class CleanRoomSession implements DurableObject {
  private state: DurableObjectState;
  private env: Env;

  constructor(state: DurableObjectState, env: Env) {
    this.state = state;
    this.env = env;
  }

  async fetch(request: Request): Promise<Response> {
    const url = new URL(request.url);

    switch (url.pathname) {
      case "/init":
        return this.initSession(request);
      case "/load-crm-data":
        return this.loadCRMDData(request);
      case "/load-partner-data":
        return this.loadPartnerData(request);
      case "/run-query":
        return this.runQuery(request);
      case "/status":
        return this.getStatus();
      case "/destroy":
        return this.destroySession();
      default:
        return new Response("Not found", { status: 404 });
    }
  }

  private async initSession(request: Request): Promise<Response> {
    const body = await request.json() as {
      partner: string;
      analysis: string;
    };
  }
}
```

```

    tenant_shop: string;
    expires_at: string;
};

await this.state.storage.put("session", {
    ...body,
    status: "initialized",
    created_at: new Date().toISOString(),
    crm_loaded: false,
    partner_loaded: false,
});

// Set alarm for auto-expiry (90 days max)
const expiresAt = new Date(body.expires_at).getTime();
await this.state.storage.setAlarm(expiresAt);

return Response.json({ ok: true, status: "initialized" });
}

private async runQuery(request: Request): Promise<Response> {
    const session = await this.state.storage.get("session") as any;
    if (!session?.crm_loaded || !session?.partner_loaded) {
        return Response.json({ ok: false, error: "Both datasets must be loaded" }, { status: 400 })
    }

    const { template } = await request.json() as { template: string };

    // Execute pre-approved query template against D1
    const result = await executeQueryTemplate(
        this.env.CLEAN_ROOM_DB,
        session.session_id || this.state.id.toString(),
        template
    );

    // Archive result to R2
    await this.env.ANALYTICS_EXPORTS.put(
        `audit/clean-room/${new Date().getFullYear()}/${session.session_id || this.state.id}/${template}.json`,
        JSON.stringify(result)
    );
}

```

```

    return Response.json(result);
  }

  // Auto-cleanup on alarm (TTL expired)
  async alarm(): Promise<void> {
    await this.destroySession();
  }

  private async destroySession(): Promise<Response> {
    const session = await this.state.storage.get("session") as any;
    const sessionId = session?.session_id || this.state.id.toString();

    // Delete all session data from D1
    await this.env.CLEAN_ROOM_DB.prepare(
      "DELETE FROM partner_uploads WHERE session_id = ?"
    ).bind(sessionId).run();

    await this.env.CLEAN_ROOM_DB.prepare(
      "DELETE FROM crm_uploads WHERE session_id = ?"
    ).bind(sessionId).run();

    // Log deletion in audit trail
    await this.env.ANALYTICS_EXPORTS.put(
      `audit/clean-room/deletions/${sessionId}.json`,
      JSON.stringify({
        session_id: sessionId,
        deleted_at: new Date().toISOString(),
        reason: "ttl_expired",
      })
    );

    await this.state.storage.deleteAll();
    return Response.json({ ok: true, status: "destroyed" });
  }
}

```

3.6 External Partner Integration

For partners that require **their own** clean room infrastructure (AWS Clean Rooms, Snowflake), the worker acts as a **clean room client** — uploading hashed data to the partner's

environment:

PARTNER	THEIR INFRASTRUCTURE	CRM SYNC ROLE
NielsenIQ Connect	AWS Clean Rooms / S3 drop	Upload hashed CSV to their S3; they run queries
Circana Unify	Snowflake Data Clean Rooms	Upload hashed JSON via API; they run queries
Google Ads Data Hub	BigQuery	Upload via GA4 Measurement Protocol; Google runs queries
Self-hosted	Cloudflare D1 (this spec)	Both datasets loaded into D1; we run queries

The Cloudflare-native clean room handles **self-hosted analysis**. Partner-hosted clean rooms use the existing analytics export delivery pipeline (see `ANALYTICS-EXPORT-SPEC.md`).

4.1 Data Classification

CLASSIFICATION	DEFINITION	EXAMPLES	CLEAN ROOM HANDLING
PII-Direct	Identifies a person on its own	Email, phone, name, address	NEVER leaves your system raw. SHA-256 hashed before any export.
PII-Indirect	Identifies when combined with other data	ZIP code, age range, gender	Allowed in clean room as attributes, not as match keys.
Pseudonymous	Hashed/tokenized identifier	SHA-256(email), CRM user_id	This is what enters the clean room. Still PII under GDPR.
Aggregated	Statistical, non-reversible	"34% match rate", "2.3x lift"	This is what exits the clean room. No longer PII.
Commercial	Business transaction data	Order total, SKU, date	Allowed in clean room, attached to pseudonymous ID.
Consent	Proof of permission	Consent record, timestamp, scope	Must accompany every record – proves lawful basis.

4.2 Immutable Security Rules

RULE 1: RAW PII NEVER LEAVES

No raw email, phone, name, or address is ever transmitted to any external system, clean room, analytics platform, or partner.

All PII is hashed (SHA-256, lowercase, trimmed) before export.

This rule has NO exceptions and NO overrides.

RULE 2: CONSENT BEFORE COMPUTATION

Every record entering a clean room must have a verifiable consent record with:

- scope: "analytics_partners" or "clean_room"
- granted_at: timestamp
- method: "explicit_optin" (not pre-checked, not bundled)
- revoked_at: NULL (not revoked)

Records without consent are excluded at the Xano query level.

The worker DOES NOT filter – Xano filters at source.

RULE 3: MINIMUM NECESSARY DATA

Only export fields required for the specific analysis.

Customer attribution: hashed_email + order_data + utm_source

Segment overlap: hashed_email + crm_tags

Category analysis: hashed_email + SKU/UPC + quantity

NEVER export: password_hash, JWT tokens, internal IDs, session data,
IP addresses, device fingerprints, browsing history

RULE 4: AGGREGATION THRESHOLD

Clean room outputs must meet minimum aggregation thresholds:

- Minimum group size: 50 individuals
- No single-record outputs allowed
- Suppress any segment with < 50 members

This prevents re-identification via small-group inference.

RULE 5: NO REVERSE ENGINEERING

Clean room queries are pre-approved and templated.

Ad-hoc queries that could enumerate individual records are blocked.
Query templates are reviewed by DPO before activation.
Neither party can export the matched record set – only aggregates.

RULE 6: TIME-BOUNDED ACCESS

Data uploaded to a clean room expires automatically:

- Analysis window: 90 days max
- Match keys: deleted after computation
- Results: retained for 12 months (aggregates only)
- Consent revocation: triggers deletion within 72 hours

RULE 7: AUDIT EVERYTHING

Every clean room operation is logged:

- Who initiated the analysis
- What data was uploaded (record count, field list, hash of dataset)
- What query was executed
- What results were returned
- When data was deleted

Audit logs are immutable (R2 archive) and retained for 5 years.

4.3 Hashing Standard

```
// CANONICAL HASHING – used across all exports and clean room uploads
// Already implemented in worker (line 2252)

async function hashPII(value: string): Promise<string> {
  const encoder = new TextEncoder();
  const data = encoder.encode(value.toLowerCase().trim()); // normalize first
  const hash = await crypto.subtle.digest("SHA-256", data);
  return Array.from(new Uint8Array(hash))
    .map(b => b.toString(16).padStart(2, "0"))
    .join("");
}

// IMPORTANT: All parties must use the SAME normalization:
// 1. Convert to lowercase
// 2. Trim whitespace
// 3. SHA-256 hash
// 4. Hex-encode (lowercase)
//
// "John@Example.com " → "john@example.com" → SHA-256 → "a1b2c3..."
//
// If NielsenIQ uses a different normalization, match rates drop to ~0%.
// Confirm normalization spec with each partner before first upload.
```


4.4 Salting Policy

CLEAN ROOM EXPORTS: NO SALT

SHA-256 without salt. This is intentional.

Both parties must produce the same hash for the same email to enable matching. Salt would make matching impossible.

This means SHA-256(email) is a pseudonymous identifier, NOT an anonymized one. It is still PII under GDPR. Consent is therefore REQUIRED (Rule 2).

INTERNAL STORAGE: SALTED

Password hashing uses PBKDF2 with random salt (line 212). These are NEVER exported. Only the unsalted SHA-256 email hash is used for clean room matching.

5. CONSENT ARCHITECTURE

5.1 New Consent Scope: `clean_room`

Added alongside existing consent scopes in `consent_records` table:

SCOPE	PURPOSE	REQUIRED FOR
tos	Terms of Service	Account creation
privacy	Privacy Policy	Account creation
marketing	Marketing communications	Email/SMS campaigns
cookie	Cookie tracking	Browser analytics
a2a	Agent-to-Agent commerce	UCP mandates
ap2	Agent-to-Person commerce	UCP agent checkout
analytics_partners	Share hashed data with measurement partners	NielsenIQ/Circana export
clean_room	Include in privacy-safe data matching	Clean room computations

5.2 Consent Collection UI

Extension Auth tab – new toggle group:

Data Sharing & Measurement

Analytics Partner Sharing

Share hashed purchase data with retail measurement partners (e.g., NielsenIQ, Circana) for aggregated market research.
Partners: [NielsenIQ] [Circana]

Privacy-Safe Data Matching (Clean Room)

Allow your hashed email to be matched against partner datasets in a secure clean room environment. No raw personal data is shared.
Your data is automatically deleted after 90 days.

5.3 Consent Verification at Export

```
-- Xano query: only export users with valid, unrevoked consent
SELECT
  u.id,
  SHA256(LOWER(TRIM(u.email))) as hashed_email,
  uc.crm_segments,
  -- order data joined separately
FROM storefront_users u
JOIN user_claims uc ON uc.user_id = u.id
JOIN consent_records cr ON cr.user_id = u.id
WHERE cr.consent_type = 'clean_room'
      AND cr.action = 'granted'
      AND cr.revoked_at IS NULL
      AND NOT EXISTS (
        -- Check for later revocation
        SELECT 1 FROM consent_records cr2
        WHERE cr2.user_id = u.id
              AND cr2.consent_type = 'clean_room'
              AND cr2.action = 'revoked'
              AND cr2.timestamp > cr.timestamp
      )
)
```

5.4 Consent Revocation Flow



6.1 Export Preparation (Xano → Worker → Clean Room)

```
XANO BACKGROUND TASK: prepare_clean_room_upload

1. Query users WHERE consent = clean_room AND NOT revoked
2. Hash all PII fields:
  • SHA-256(lowercase(trim(email)))
  • SHA-256(phone) – if consented and available
3. Attach non-PII attributes:
  • CRM segment tags (lifestyle, preferences)
  • Order history (UPCs, quantities, dates, totals)
  • Channel source (utm_source, utm_medium)
  • UCP signals (mandate_used: true/false)
4. Validate: ensure no raw PII in output
5. Generate manifest:
  • Record count
  • Field list
  • SHA-256 of entire dataset (integrity check)
  • Consent proof summary (count by scope, oldest grant date)
6. Insert into integration_queue (status: ready)
7. POST webhook → Worker /events/deliver
```

6.2 Upload Record Schema

```
interface CleanRoomRecord {
  // Match keys (hashed – the ONLY way to link across datasets)
  hashed_email: string;           // SHA-256 hex
  hashed_phone?: string;          // SHA-256 hex, optional

  // Attributes (non-PII, attached to match key)
  segments: string[];             // ["health_conscious", "subscriber", "high_ltv"]
  order_count_30d: number;        // Orders in last 30 days
  order_total_30d: number;        // Revenue in last 30 days
  avg_order_value: number;
  product_categories: string[];   // ["supplements", "protein", "skincare"]
  upcs_purchased: string[];       // Mapped from Shopify SKU → UPC
  channel_source: string;         // "google_ads" | "organic" | "email" | "social" | "agent_ucp"
  has_mandate: boolean;           // UCP agent mandate signal
  first_purchase_date: string;    // ISO date
  customer_lifetime_days: number;

  // Consent proof (travels with the record)
  consent_scope: string;          // "clean_room"
  consent_granted_at: string;     // ISO timestamp
  consent_method: string;         // "explicit_optin"

  // Metadata
  export_date: string;            // ISO date
  tenant_shop: string;            // Multi-tenant scoping
}

// VERIFICATION: Before upload, scan every record:
// - hashed_email matches /^[a-f0-9]{64}$/
// - No field contains "@" (email leak check)
// - No field contains a phone pattern /\d{10,}/
// - No field contains a name pattern (NER check optional)
// - consent_granted_at is a valid timestamp and is before export_date
```

6.3 PII Leak Prevention

```
// Run BEFORE any clean room upload – last line of defense

function validateCleanRoomPayload(records: CleanRoomRecord[]): { valid: boolean; violations: string[] } {
  const violations: string[] = [];

  for (let i = 0; i < records.length; i++) {
    const r = records[i];
    const prefix = `Record ${i}`;

    // Hash format check
    if (!/^[a-f0-9]{64}$/.test(r.hashed_email)) {
      violations.push(`${prefix}: hashed_email is not a valid SHA-256 hash`);
    }
    if (r.hashed_phone && /^[a-f0-9]{64}$/.test(r.hashed_phone)) {
      violations.push(`${prefix}: hashed_phone is not a valid SHA-256 hash`);
    }

    // Scan all string fields for PII leaks
    const allStrings = JSON.stringify(r);

    // Email pattern
    if (/@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/.test(allStrings)) {
      violations.push(`${prefix}: contains email pattern – raw PII leak`);
    }

    // Phone pattern (10+ consecutive digits)
    if (/\\d{10,}/.test(allStrings.replace(/^[a-f0-9]{64}/g, ""))) { // exclude hashes
      violations.push(`${prefix}: contains phone number pattern`);
    }

    // Name-like patterns in non-segment fields
    // (segments are allowed to contain words like "health_conscious")

    // Consent check
    if (!r.consent_scope || !r.consent_granted_at) {
      violations.push(`${prefix}: missing consent proof`);
    }
  }
}
```

```
    if (r.consent_granted_at > r.export_date) {
      violations.push(`${prefix}: consent granted AFTER export - temporal violation`);
    }
  }

  // Aggregation threshold - minimum 50 records
  if (records.length > 0 && records.length < 50) {
    violations.push(`Dataset has ${records.length} records - below minimum threshold of 50`);
  }

  return { valid: violations.length === 0, violations };
}

// ENFORCEMENT: If validation fails, the upload is BLOCKED.
// No override. No manual approval. Fix the data and re-run.
```

7. CLEAN ROOM QUERY TEMPLATES

Pre-approved queries — ad-hoc queries are **not allowed**.

Q1: Attribution Lift

Question: "Did customers who saw our Google Ad and bought D2C also buy in-store?"


```
-- Runs INSIDE the clean room (neither party sees raw data)
SELECT
  crm.channel_source,
  COUNT(DISTINCT crm.hash_email) as crm_customers,
  COUNT(DISTINCT CASE WHEN retail.hash_email IS NOT NULL
    THEN crm.hash_email END) as also_bought_retail,
  ROUND(100.0 * COUNT(DISTINCT CASE WHEN retail.hash_email IS NOT NULL
    THEN crm.hash_email END) / COUNT(DISTINCT crm.hash_email), 1) as match_pct,
  AVG(CASE WHEN retail.hash_email IS NOT NULL
    THEN retail.units_sold END) as avg_retail_units
FROM crm_upload crm
LEFT JOIN retail_data retail ON crm.hash_email = retail.hash_email
GROUP BY crm.channel_source
HAVING COUNT(DISTINCT crm.hash_email) >= 50 -- aggregation threshold
```

Output (what you receive):

CHANNEL_SOURCE	CRM_CUSTOMERS	ALSO_BOUGHT_RETAIL	MATCH_PCT	AVG_RETAIL_UNITS
google_ads	2,340	798	34.1%	4.2
organic	5,120	1,024	20.0%	2.8
agent_ucp	156	72	46.2%	6.1
email	890	267	30.0%	3.5

Q2: Segment Overlap

Question: "Which CRM segments have the highest retail purchase overlap?"

```
SELECT
  UNNEST(crm.segments) as segment,
  COUNT(DISTINCT crm.hashcd_email) as segment_size,
  COUNT(DISTINCT CASE WHEN retail.hashcd_email IS NOT NULL
    THEN crm.hashcd_email END) as retail_match,
  AVG(retail.total_spend) as avg_retail_spend
FROM crm_upload crm
LEFT JOIN retail_data retail ON crm.hashcd_email = retail.hashcd_email
GROUP BY UNNEST(crm.segments)
HAVING COUNT(DISTINCT crm.hashcd_email) >= 50
ORDER BY avg_retail_spend DESC
```

Q3: UCP Mandate Impact

Question: "Do customers who used AI agent mandates spend more in-store?"

```
SELECT
  crm.has_mandate,
  COUNT(DISTINCT crm.hashcd_email) as customers,
  AVG(crm.order_total_30d) as avg_dtc_spend,
  AVG(retail.total_spend) as avg_retail_spend,
  AVG(crm.order_total_30d + COALESCE(retail.total_spend, 0)) as avg_total_spend
FROM crm_upload crm
LEFT JOIN retail_data retail ON crm.hashcd_email = retail.hashcd_email
GROUP BY crm.has_mandate
HAVING COUNT(DISTINCT crm.hashcd_email) >= 50
```

Q4: Google Ads Offline Conversion (for import back to Google)

```
-- Output format compatible with Google Ads offline conversion import
SELECT
  crm.hashed_email,
  retail.purchase_date as conversion_date,
  retail.total_spend as conversion_value,
  'in_store_purchase' as conversion_action
FROM crm_upload crm
JOIN retail_data retail ON crm.hashed_email = retail.hashed_email
WHERE crm.channel_source = 'google_ads'
      AND retail.purchase_date BETWEEN crm.first_purchase_date
      AND DATE_ADD(crm.first_purchase_date, INTERVAL 30 DAY)
-- NOTE: This outputs hashed_email (not raw) – Google matches
-- against their own hash of the user's Google account email
```

Worker: Clean Room Session Endpoints

```
POST /admin/clean-room/session
```

```
Authorization: Bearer {ADMIN_KEY}
```

```
Body: {
```

```
  "partner": "nielseniq",           // or "circana", "google_adh", "self"
```

```
  "analysis": "attribution_lift",    // pre-approved query template
```

```
  "date_range": { "from": "2026-04-01", "to": "2026-05-27" },
```

```
  "shop": "hx-stage.myshopify.com"  // tenant scoping
```

```
}
```

```
Response: {
```

```
  "ok": true,
```

```
  "session_id": "cru_a1b2c3d4",
```

```
  "records": 4821,
```

```
  "records_excluded_no_consent": 1203,
```

```
  "pii_validation": "passed",
```

```
  "manifest_hash": "sha256:e5f6g7...",
```

```
  "destination": "d1://crm-clean-room/session/cru_a1b2c3d4",
```

```
  "expires_at": "2026-08-25T00:00:00Z"
```

```
}
```

POST /admin/clean-room/partner-upload

Authorization: Bearer {ADMIN_KEY}

Content-Type: application/json

Body: {

 "session_id": "cru_a1b2c3d4",

 "partner": "nielseniq",

 "records": [

 { "hashed_email": "a1b2c3...", "attributes": { "units_sold": 4, ... } }

]

}

Response: {

 "ok": true,

 "records_loaded": 12450,

 "session_status": "partner_loaded"

}

POST /admin/clean-room/query

Authorization: Bearer {ADMIN_KEY}

Body: {

 "session_id": "cru_a1b2c3d4",

 "template": "attribution_lift" // pre-approved template only

}

Response: {

 "session_id": "cru_a1b2c3d4",

 "status": "completed",

 "analysis": "attribution_lift",

 "results": { ... }, // aggregated, non-PII outputs

 "computed_at": "2026-05-27T14:30:00Z",

 "match_rate": 0.341,

 "records_matched": 1644,

 "records_total": 4821,

 "aggregation_threshold_met": true

}

```
GET /admin/clean-room/results?session_id=cru_a1b2c3d4
```

```
Authorization: Bearer {ADMIN_KEY}
```

```
Response: {
```

```
  "session_id": "cru_a1b2c3d4",
```

```
  "status": "completed",
```

```
  "queries": [ ... ],           // all query results for this session
```

```
  "data_expires_at": "2026-08-25T00:00:00Z",
```

```
  "audit_url": "r2://audit/clean-room/2026/cru_a1b2c3d4/"
```

```
}
```

```
DELETE /admin/clean-room/session?session_id=cru_a1b2c3d4
```

```
Authorization: Bearer {ADMIN_KEY}
```

```
Response: {
```

```
  "ok": true,
```

```
  "session_id": "cru_a1b2c3d4",
```

```
  "status": "destroyed",
```

```
  "records_deleted": { "crm": 4821, "partner": 12450, "results": 3 }
```

```
}
```

D1 Query Execution (replaces external clean room compute)

```
// Pre-approved query templates executed against D1
async function executeQueryTemplate(
  db: D1Database,
  sessionId: string,
  template: string
): Promise<QueryResult> {
  const queries: Record<string, string> = {
    attribution_lift: `
      SELECT
        c.channel_source,
        COUNT(DISTINCT c.hashed_email) as crm_customers,
        COUNT(DISTINCT CASE WHEN p.hashed_email IS NOT NULL
          THEN c.hashed_email END) as also_bought_retail,
        ROUND(100.0 * COUNT(DISTINCT CASE WHEN p.hashed_email IS NOT NULL
          THEN c.hashed_email END) / COUNT(DISTINCT c.hashed_email), 1) as match_pct
      FROM crm_uploads c
      LEFT JOIN partner_uploads p ON c.hashed_email = p.hashed_email
      AND c.session_id = p.session_id
      WHERE c.session_id = ?1
      GROUP BY c.channel_source
      HAVING COUNT(DISTINCT c.hashed_email) >= 50
    `,
    segment_overlap: `
      SELECT
        json_each.value as segment,
        COUNT(DISTINCT c.hashed_email) as segment_size,
        COUNT(DISTINCT CASE WHEN p.hashed_email IS NOT NULL
          THEN c.hashed_email END) as retail_match
      FROM crm_uploads c, json_each(c.segments)
      LEFT JOIN partner_uploads p ON c.hashed_email = p.hashed_email
      AND c.session_id = p.session_id
      WHERE c.session_id = ?1
      GROUP BY json_each.value
      HAVING COUNT(DISTINCT c.hashed_email) >= 50
    `,
    // ... additional templates
  };
};
```

```
const sql = queries[template];
if (!sql) throw new Error(`Unknown query template: ${template}`);

const result = await db.prepare(sql).bind(sessionId).all();
return {
  template,
  rows: result.results,
  aggregation_threshold_met: true,
  computed_at: new Date().toISOString(),
};
}
```

Xano: Clean Room Preparation Task

Same pattern as analytics export — Xano does the heavy lifting:

1. Query users with clean_room consent (unrevoked)
2. Join order data, CRM tags, UCP mandate history
3. Map Shopify SKUs → UPCs
4. Hash all PII (hashPII function – shared normalization)
5. Run PII leak validation
6. Format as JSON for D1 insert (self-hosted) or CSV/Parquet (partner-hosted)
7. Insert into integration_queue → webhook to Worker
8. Worker loads data into D1 (self-hosted) or uploads to partner S3/API (partner-hosted)

9. AUDIT TRAIL

Every clean room operation generates an immutable audit record:


```
interface CleanRoomAuditEntry {
    audit_id: string;                // UUID
    operation: "upload" | "query" | "result" | "delete" | "consent_revocation";
    session_id: string;
    provider: string;                // "cloudflare_d1" | "partner_s3" | "partner_api" | "goc
    tenant_shop: string;
    initiated_by: string;            // admin key preview or "system_cron"

    // Upload details
    record_count?: number;
    fields_included?: string[];      // ["hashed_email", "segments", "order_count_30d"]
    fields_excluded?: string[];      // ["raw_email" - never included but logged as proof]
    consent_summary?: {
        total_users: number;
        consented_users: number;
        excluded_no_consent: number;
    };
    dataset_hash?: string;           // SHA-256 of entire upload - integrity proof

    // Query details
    query_template?: string;         // "attribution_lift" - only pre-approved templates

    // Result details
    match_rate?: number;
    aggregation_threshold_met?: boolean;

    // Timing
    timestamp: string;
    data_expires_at?: string;

    // Compliance
    legal_basis: string;             // "consent" - Art. 6(1)(a) GDPR
    dpo_approved_template: boolean;
}

// Storage: R2 archive (immutable, 5-year retention)
// Path: audit/clean-room/{year}/{month}/{audit_id}.json
```

10. STAKEHOLDER ACCESS

CAPABILITY	A (CREATOR)	B (SHARED)	C (PRIVATE)
Configure clean room provider	✓	×	✓ Own account
Trigger upload	✓	×	✓
View results	✓	×	✓
Approve query templates	✓ (DPO role)	×	✓ (their DPO)
Manage consent toggles	✓ Extension	✓ Extension	✓ Extension
View audit trail	✓	×	✓
Delete clean room data	✓	×	✓
Consent revocation propagation	✓ Auto	✓ Auto	✓ Auto

Scenario: Clean room data breach at partner

1. IMMEDIATE (0-1 hours)

- └ Revoke partner API credentials (KV delete)
- └ DELETE FROM partner_uploads / crm_uploads WHERE session involves partner
- └ Destroy Durable Object sessions for affected partner
- └ Log incident in R2 audit trail
- └ Notify DPO

2. ASSESSMENT (1-24 hours)

- └ Determine what data was exposed
- └ Data was hashed - assess re-identification risk
- └ If hashed emails + other attributes could enable
 - | re-identification → treat as PII breach
- └ Document in incident report

3. NOTIFICATION (24-72 hours)

- └ If GDPR applies and risk to individuals:
 - | Notify supervisory authority within 72 hours
- └ If high risk to individuals:
 - | Notify affected data subjects
- └ Log notification in audit trail

4. REMEDIATION

- └ Review partner's security posture
- └ Consider partner suspension
- └ Review aggregation thresholds
- └ Update DPO-approved query templates if needed

Scenario: User requests deletion (GDPR Art. 17)

1. User submits deletion request via POST /auth/delete-account
2. Worker processes account deletion (existing flow)
3. ADDITIONALLY:
 - └ Query D1: SELECT session_id FROM crm_uploads WHERE hashed_email = ?
 - └ For each session with matching records:
 - | DELETE FROM crm_uploads WHERE hashed_email = ? AND session_id = ?
 - | DELETE FROM partner_uploads WHERE hashed_email = ? AND session_id = ?
 - └ For partner-hosted clean rooms:
 - | POST {partner}/api/delete-records { match_keys: [hashed_email] }
 - └ Log all deletions in R2 audit trail
 - └ Confirm deletion within 72 hours
4. Cron cleanup: verify D1 deletion complete + partner confirmations received